How MLTorq runs

A plain-language owner's guide to MLTorq by GearOps, built on the RivetCI engine

For Sudhakar Chennoju, founder. Prepared Oct 31, 2026 (IST) from the code on branch pipeline-run (HEAD 3ad8521).

This guide contains no secret values. It names secrets (for example DB_PASSWORD) so you know where they live, but never shows what they are. Every fact was checked against the repo files named in brackets, such as [deploy/up.sh]. Anything I could not confirm is marked "Verify" and collected in the "Verify before the demo" list at the end.

Contents

1. Elevator explanation (one paragraph, 30 seconds, 2 minutes)
2. Architecture and what each part does
3. Life of a request and life of a job
4. Task types: the core idea (the moat)
5. Pipelines, jobs, runners, environments, targets
6. The demo pipelines and what to say
7. Deploy and operations
8. Secrets and security
9. GitHub integration end to end
10. Honest limitations and roadmap
11. Likely investor and interviewer questions
12. Glossary
13. Cheat sheet of commands
14. Verify before the demo

Names in one line: GearOps is the company. MLTorq is the product customers see (mltorq.com). RivetCI is the CI/CD engine inside it that you built from scratch. Java package names still say "cicada" (orchestrator) and "devxp" (spinner); those are old internal names, not separate products.

1. The elevator explanation

One paragraph

MLTorq is CI/CD for the AI lifecycle. Today a team builds code in one tool, fine-tunes in another, evaluates in a third and picks a model in a fourth, and no single run can say "this model passed quality and security, here is what it cost, ship it". MLTorq puts all of those steps into one pipeline on one engine. Every step is a "task type": a small script plus a form. Building code, scanning for leaked keys, LoRA fine-tuning, scoring an LLM, choosing the cheapest model that still passes, and deploying to AWS, Azure, GCP or your own servers are all task types. A gate step can stop the pipeline, so a regressed model or a vulnerable build never reaches deploy. It runs in the customer's own infrastructure on Docker, Kubernetes or AWS ECS, with one secrets store, one log trail and one job page.

30-second spoken version

Say it: *AI teams ship models through four or five disconnected tools: CI for code, a notebook or GPU box for fine-tuning, an eval tool, and a gateway to pick models. None of them shares a run, so nobody can say "this exact model passed and cost this much". MLTorq is one CI/CD engine for all of it. Every step, from a security scan to a LoRA fine-tune to "pick the cheapest model that still passes", is a task in one pipeline, with gates that block a bad model before deploy. It runs in your own cloud. I built the engine, RivetCI, from scratch.*

2-minute spoken version

Say it: *Here is the problem. A company wants to swap an expensive model for a cheaper one, or ship a fine-tuned model. The code goes through CI. The fine-tune runs on someone's GPU box. The evaluation lives in another tool. The cost is a spreadsheet. Each has its own secrets and logs and its own idea of "passed", so a regressed model can slip into production and nobody can say what a run cost.*

Say it: *MLTorq is CI/CD built for that whole lifecycle. A pipeline is a chain of tasks. Each task is a script with a form: you fill in the form, the engine fills the script and runs it in a fresh container. Because everything is a task, a security scan, a LoRA fine-tune, an eval gate and a deploy all look the same to the engine. They share one secrets store, one log trail and one job page.*

Say it: *Let me show three examples. First, the same fine-tune pipeline with good data passes the eval gate; with noisy data it is blocked, and deploy never runs. Second, the same security pipeline blocks a repo with leaked keys and vulnerable libraries, and passes the clean one. Third, our routing task sends 50 labelled support tickets to Haiku, Sonnet and Opus, scores each, and picks the cheapest model that clears the bar. Here, Haiku scored 1.00, as good as Sonnet, so we ship the cheap one.*

Say it: *Under the hood is RivetCI, an engine I wrote from scratch: a Spring Boot orchestrator, a spinner that launches jobs on Docker, Kubernetes or AWS ECS, and Kafka between them. It deploys to 15-plus targets across AWS, Azure, GCP and on-prem. It runs in the customer's own account, so their data and GPUs stay with them. The point tools in this space are being bought by bigger platforms; we want to be the independent engine that owns the gate before deploy.*

2. Architecture

Everything for the hosted demo runs on one AWS EC2 virtual machine (VM) in us-east-1 (N. Virginia), AWS account 174565890211, Elastic IP 54.197.145.44, using Docker Compose. Docker Compose is a tool that starts several containers together from one file [deploy/docker-compose.vm.yml]. Only the web proxy (Caddy) is reachable from the internet; the database and Kafka are private to the VM.

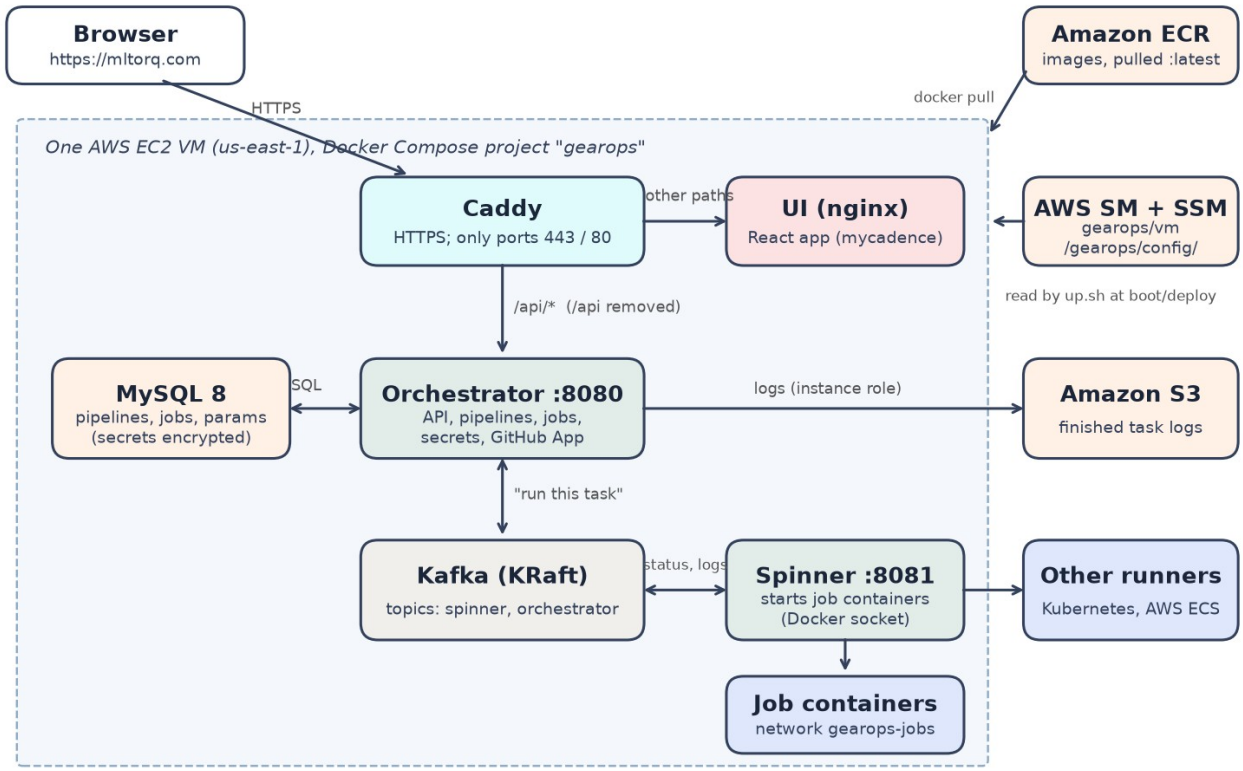


Figure 1. The production demo stack on one VM. Arrows show who talks to whom.

Each component and its job

Component	What it is (one line)	Its job in MLTorq
Caddy (caddy:2-alpine)	A web server that also gets HTTPS certificates automatically (Let's Encrypt).	The front door. Serves HTTPS on 443 (and 80 only to redirect and for certificate checks). Sends /api/* to the orchestrator with "/api" removed, and everything else to the UI [deploy/caddy/Caddyfile.vm].
UI (gearops-ui, nginx)	The React web app, source folder mycadence/, served by nginx.	Pipeline canvas, task forms, job page with live logs, Repositories, Targets, Settings. It calls the API at <your site>/api/... [mycadence/src/apibase.js].
Orchestrator (gearops-orchestrator, Spring Boot, port 8080)	The brain. A Java Spring Boot service.	Login (JWT tokens), projects, pipelines, task types, jobs, secrets, deploy targets, cost plans, GitHub App. Turns a pipeline into job tasks, fills in form values, resolves secrets, sends tasks to the spinner over Kafka, collects logs, stores finished logs in S3.
Spinner (gearops-spinner, Spring Boot, port 8081)	The hands. A Java service that launches work.	Receives tasks from Kafka, builds the final script, and runs it on a runner: Docker on this VM, a Kubernetes cluster, or AWS ECS

		[spinner/.../builder: DockerPipelineRunner, KubePod, EcsTaskRunner]. Streams logs and status back.
Kafka (apache/kafka 3.9, KRaft mode)	A message queue: services drop messages in, others pick them up.	Carries messages between orchestrator and spinner. Topic "spinner" = work for the spinner; topic "orchestrator" = results and logs back [application.properties: listen.topic/publish.topic]. KRaft means no separate ZooKeeper.
MySQL 8 (database gearml)	The database.	Stores users, projects, pipelines, steps (pipeline_task_type), form answers (param), task types and their fields, jobs, deploy targets, cost data. Secret values are stored encrypted (ENCv1).
keystore-init (one-shot)	A tiny job that runs once at startup.	Creates the AES encryption keystore (alias ff-aes-v1) used to encrypt secrets, only if it does not exist yet.
Job containers	Short-lived containers, one per job, deleted when done.	Where the user's scripts actually run. On the VM they join a separate network "gearops-jobs" that cannot reach the app or the AWS metadata endpoint.
Amazon ECR	AWS's private container image store.	Holds the three images gearops-orchestrator, gearops-spinner and gearops-ui. The VM pulls the tag in IMAGE_TAG (latest).
AWS Secrets Manager + SSM Parameter Store	AWS services for secrets (encrypted) and settings.	Secret gearops/vm holds the passwords and keys; /gearops/config/ holds settings like GEAROPS_DOMAIN and IMAGE_TAG. Read only by up.sh on the VM.
Amazon S3 bucket gearops-deploy-174565890211	AWS file storage.	Finished task logs under logs/. Written with the VM's role, so no keys are needed.

Verify: the deck (slide 4) says "Postgres". The code uses MySQL 8. Fix the slide or say "MySQL" in the demo.

Why it is built this way (good lines for interviews)

- **Split brain and hands.** The orchestrator decides; the spinner executes. They only talk through Kafka, so later the spinner can sit inside a customer's network while the orchestrator stays with us (the "runner agent" on the roadmap).
- **One engine, many runners.** The same task runs on Docker, Kubernetes or ECS. The choice is made when you start the job, not when you write the pipeline.
- **Ephemeral containers.** Every job gets a fresh container that is thrown away. No leftover state, no "works on my build box".
- **Small footprint.** The whole control plane fits on one 2 vCPU / 8 GB VM for about $70 a month.

3. Life of a request and life of a job

Life of a request (you click something in the UI)

1. Your browser calls https://mltorq.com/api/orchestrate/... with your login token (a JWT, a signed token that proves who you are).
2. Caddy receives it on port 443, sees the path starts with /api/, removes "/api", and forwards it to orchestrator:8080 as /orchestrate/....
3. The orchestrator checks the token, does the work (usually reading or writing MySQL), and returns JSON.
4. Any other path (/, /pipelines, /repos ...) goes to the UI container, which returns the React app. The app then makes its own /api calls.

Life of a job (you press Run on a pipeline)

1. Start. The UI calls the orchestrator to start the pipeline, and you pick a runner (for the demo, "This server (Docker)").
2. Copy steps into a job. JobService creates a job and one job task per pipeline step, in the saved order.
3. Fill the form values. Blank fields get their default values. Secret references like ${secret:ANTHROPIC_API_KEY} are swapped for the real value only at this point (SecretRuntimeService). Each real secret is registered with SecretMasker so it shows as **** if it ever appears in a log.
4. Send to Kafka. The orchestrator publishes "run this task" on topic "spinner". Secret params stay encrypted in the message.
5. Spinner picks it up. CommandResolver decrypts the encrypted values and fills the ${field} placeholders in the script. Secret fields are delivered as environment variables named SECRET_<field> rather than pasted into the script text.
6. Run. For the Docker runner, DockerPipelineRunner starts a fresh container (for the demos python:3.11-slim) on network gearops-jobs, with CPU, memory, process and time limits, no host folders, no Docker socket and no app secrets. One job runs at a time on the VM; others queue.
7. Logs. Each output line flows back through Kafka to the orchestrator, which streams it to your browser live (server-sent events, a one-way live feed) and buffers it on disk.
8. Gate. If a script exits with a non-zero code (for example the Eval Gate says FAIL), the task fails and every later step is skipped, including deploy. That is how "BLOCKED" works.
9. Finish. Finished task logs are uploaded to S3 (logs/ prefix), metrics printed as GEAROPS_METRIC lines are stored, and the container is deleted.

Say it: *The engine never trusts the job. The job runs in a throwaway container on its own network, gets only the secrets its form asked for, and if it prints one, it is masked.*

4. Task types: the core idea, and why it is the moat

A task type is a recipe. It is a shell script with placeholders like ${host} or ${threshold}, plus a list of fields that describe each placeholder. MLTorq turns those fields into a form automatically. A pipeline step is the recipe used once, with the answers filled in. A job is one run of the pipeline [docs/learning/task-types.md].

Thing	Stored in	Example
Task type (recipe)	table task_type: name, command (the script), is_builtin, target_resource_type_key ...	"LLM: Cheapest passing model" (id 931)
Field (one form question)	table task_type_field: field_name, field_type (text, textarea, select), label, help_text, is_encrypted, is_required, default_value, options	threshold, default 0.85
Step (recipe used once)	table pipeline_task_type; its answers in table param	pipeline 398, step 2
Job (one run)	job tables	one run of pipeline 398

How a placeholder becomes a form field

1. The script contains ${threshold}.
2. There is a field row with field_name = threshold, type text, label "Threshold", default 0.85.
3. The UI draws a text box labelled "Threshold" with that hint. A select field becomes a drop-down; a field with is_encrypted = 1 becomes a locked secret box.
4. When you save, each answer becomes a param row. Secret answers are encrypted before saving and are never sent back to the browser.
5. At run time the engine swaps ${threshold} for your answer and runs the script.

A validator (ScriptPlaceholderValidator, mirrored in scripts/gen_deploy_task_types.py) checks every built-in script before it ships: every ${name} must be a declared field, secrets must be read as $SECRET_<field>, and values in locked built-ins must be one line with no shell-special characters. Built-in task types are locked: the runner always uses the database copy of the script, so a user cannot swap it.

Three concrete examples from the code

- **LLM: Cheapest passing model (id 931).** 14 fields, including provider (Anthropic, OpenAI-compatible or Mock), mode (stop at first pass, or compare all), candidates (cheapest first), evalFile (labelled tickets), metric (accuracy or macro F1), threshold, promptTemplate, apiKey (a reference ${secret:ANTHROPIC_API_KEY}, not the key), maxCalls (spend cap) and reportPath. It runs every candidate on the same tickets and picks the cheapest one that passes.
- **Deploy to EC2 (typeKey ec2).** Fields region, instanceId, host, username, sshKey (secret), port, remoteDir, artifactDir, runtime, runCommand, appPort, runMode, appSecrets. Script orchestrator/src/main/resources/scripts/deploy/ec2.sh, defined in manifest.json. It can be filled from a saved Deploy Target.
- **Security gates (ids 911 to 914).** Secret Scan (Gitleaks), Dependency Scan, Code Scan (Semgrep), Container Image Scan (Trivy). Fields: code folder, fail threshold (HIGH), fail on findings. A HIGH finding makes the script exit 1, which blocks the deploy.

Other built-ins: LLM Fine-tune (926, LoRA with transformers + peft), LLM Eval (928), Eval Gate (924), Download model weights (927), the six Titanic MLGear steps (917 to 922), Drift check PSI (923), Demo files (samples), and 16 deploy types across AWS (EC2, Lambda, ECS, EKS, S3 static site), Azure (VM, Functions, AKS, Container Apps), GCP (Compute, Cloud Run, GKE) and on-prem (SSH, Docker host, Kubernetes).

Why task types are the moat

- **Every new capability is just another task type.** Fine-tune, eval, routing, scans and deploys did not need new engine code paths. They are scripts plus forms. That is why one person could ship this much.
- **Everything shares the plumbing for free.** Each new task automatically gets secrets handling, masking, logs, gates, cost metrics, runners and the job page.
- **Customers and the LLM can add their own.** Users write custom task types, or have an LLM draft one, and they behave exactly like built-ins. The AI pipeline builder can only suggest task types the engine already knows how to run (hidden_from_ai controls this).
- **The rule you hold yourself to.** Every feature must fit the task type model. If it does not fit, it waits. That discipline keeps the engine small.

Say it: *In MLTorq, everything is a task: a script plus a form. Adding fine-tuning or model routing did not change the engine. It added a task. That is why one engine can cover CI, ML training, evaluation and deploy, and why each new task inherits secrets, logs, gates and cost tracking for free.*

5. Pipelines, jobs, runners, environments, targets

Concept	Plain meaning	Notes
Project	A folder for related pipelines.	Pipelines live in a project.
Pipeline	An ordered chain of steps, drawn on the canvas.	Steps run in the saved order (taskTypeOrder, then row id; commit 3ad8521). A hidden step has order -1.
Job	One run of a pipeline.	Has a status, a task list, live logs, metrics.
Runner	Where the job's containers run.	Kinds: Docker ("This server (Docker)", on the VM), Kubernetes (EKS, AKS, minikube…), AWS ECS. Picked when you start a job.
"This server (Docker)" runner	The built-in runner seeded on first start.	1.5 CPU, 2048 MB memory, 30-minute timeout, 512 process limit, one job at a time [docker-compose.vm.yml].
Environment	A reusable set of settings for a step, e.g. "JDK 21".	There is no default environment any more; you pick one.
Deploy Target	A saved place to deploy (an EC2 host, a cluster…), with its settings encrypted.	A deploy step can prefill its fields from a saved target, or "Save as target". Stored in the resource table.
Secret	A named value on Settings > Secrets, referenced as ${secret:NAME}.	E.g. ANTHROPIC_API_KEY, DEMO_TARGET_SSH_KEY. Stored encrypted.
Cost plan	A GPU budget with an alert and a stop rule.	MLGear > Cost plans. A bound job can be stopped when the budget runs out (CostGuardSweeper).
Starter pipeline (template)	A read-only recipe for a whole pipeline.	Spring Boot CI, Titanic (MLGear), LLM fine-tune + eval gate. Click Use to copy it.

Important: model serving in the Titanic demo ("Deploy model API", MLGear) needs a Kubernetes runner. Serving on the Docker runner is not supported.

6. The demo pipelines and what to say

Pipeline ids are from the live VM database. The seed script [deploy/laptop/seed-demo-data.ps1] runs them as the "demo" user. It describes each as starting with a Public Git Clone of the public repo github.com/gearops/mltorq-demos, with the "Demo files (samples)" task as an offline fallback. On 399, the clone step is hidden (order -1) and Demo files supplies the inputs.

Id	Name	What it does (one line)	Expected
391	LLM fine-tune demo 1	LoRA fine-tune of a tiny 135M model on CPU (smoke mode), then LLM Eval gate.	PASSED (base 0.00 -> tuned 1.00)
392	LLM fine-tune demo 2	Same pipeline, noisy data (wrong answers).	BLOCKED by eval gate
393	Security scan demo 1	Gitleaks, dependency scan, Semgrep, Trivy on a vulnerable sample (fake keys generated at run time).	BLOCKED
394	Security scan demo 2	Same scans on a clean sample.	PASSED, deploy echo runs
395	Support ticket triage 1	LLM routing with the Mock provider. Rehearsal, no API cost.	Completes
396 / 397	Support ticket triage 2 / 3	Real Anthropic calls, stop at first passing model.	Completes
398	Support ticket triage 4	Compare all: Haiku 1.00, Sonnet 1.00, Opus 0.98 on 50 labelled tickets.	Haiku chosen (cheapest that passes)
399	MLGear Titanic train, gate & deploy	Load, train, evaluate, register, promotion gate, deploy model API.	Promoted (serving needs Kubernetes)
400	Titanic drift check	One task, Drift check (PSI). PSI > 0.2 means drift, which triggers a retrain.	Reports drift (one task by design)
401	Spring Boot CI demo	Clones spring-petclinic, Maven build, secret + image scan, then Deploy to EC2 on the free-tier VM gearops-demo-target (Deploy Target 3).	Deployed; needs secret DEMO_TARGET_SSH_KEY

What to say for each

- **Fine-tune gate (391 vs 392).** "Same pipeline, only the data changed. Good data: the fine-tuned model beats the base model and ships. Noisy data: the eval gate blocks it and deploy never runs. No GPU, no API cost."
- **Security gate (393 vs 394).** "Four scanners run as gates. The vulnerable repo has leaked keys and old libraries, so it is blocked before deploy. The keys are fake, generated at run time."
- **LLM cost routing (398).** "Fifty labelled support tickets went to Haiku, Sonnet and Opus. Haiku scored 1.00, the same as Sonnet. The pipeline picks the cheapest model that clears the bar, so we would ship Haiku." Do not quote a dollar figure: no dollar cost is stored for these runs.
- **Titanic MLOps (399, 400).** "Classic ML too: train, evaluate, register, and a promotion gate that only promotes a model that beats production. A drift check watches the data and triggers retraining."
- **Spring Boot CI (401).** "And it is still a normal CI/CD engine: build, test, scan and deploy a Java app to a VM."

Mismatch to flag: you decided on 5 merged demo pipelines (fine-tune with gate, security scan, LLM cost routing, Titanic, Spring Boot CI). I found no merge in the repo: the seed script still lists the separate ids 391 to 401, and the deck slide 6 shows the pass/blocked pairs. Either the merge happened only in the live database, or it has not been done yet. Check the Pipelines page before recording.

7. Deploy and operations

The flow

1. Code. You change code on the laptop in C:\projects\gearops-deploy on branch pipeline-run, commit, then fast-forward develop to the same commit. Never main.
2. Build. Build the Docker images for linux/amd64 from deploy/orchestrator.Dockerfile, deploy/spinner.Dockerfile, deploy/ui.Dockerfile (images build with tests skipped, because one spinner test does not compile).
3. Push. Tag and push to ECR: 174565890211.dkr.ecr.us-east-1.amazonaws.com/gearops-orchestrator (and -spinner, -ui), tagged with the git commit and latest.
4. Deploy on the VM. Open a shell with SSM (no SSH port is open), then run deploy/up.sh. It reads secrets and config from AWS, writes the env file, logs in to ECR, then "docker compose pull" and "up -d". Only containers whose image changed restart.
5. On every reboot, systemd service gearops.service runs up.sh again, because the env file lives in memory (/run) and is gone after a reboot.

Verify: I found no committed build-and-push script in the repo. The build steps are in docs/deploy-ec2-plan.md, and the agent ran them by hand. Note that the plan doc names the repos gearops/<x>, while the compose file uses gearops-<x>; the compose file is what runs.

Currently deployed (from the session notes)

Image	Tag	Note
UI	3ad8521	rollback: 3793bf7
Orchestrator	271f40a	not rebuilt yet, so the list-order fix in 3ad8521 is not live there
Spinner	6614ef3	

Health checks

- From anywhere: open https://mltorq.com and log in; or call https://mltorq.com/api/orchestrate/edition (the health check the containers use, since /actuator/health needs login).
- Laptop: .\gearops-vm.ps1 status shows the VM state and IP.
- On the VM: docker compose -f docker-compose.vm.yml --env-file /run/gearops/gearops.env ps shows each container; add "logs --tail 100 orchestrator" for logs.

Rollback

Images are tagged per commit, so rollback means pointing at the previous tag. Either retag the previous image as latest in ECR and run up.sh again, or set SSM /gearops/config/IMAGE_TAG to the old tag (note: it applies to all three images). The database is not rolled back by this. Back it up first with deploy/backup.sh.

Where the logs are

- Job logs while running: buffered in the "logs" Docker volume (/app/data/logs) and streamed live to the job page.
- Finished task logs: S3 bucket gearops-deploy-174565890211, prefix logs/, us-east-1.
- Service logs: docker compose logs <service> on the VM.
- Backups: deploy/backup.sh writes a database dump plus the keystore to /var/backups/gearops (keeps 7), and to S3 if BACKUP_S3_URI is set. Without the keystore backup, encrypted secrets in a dump cannot be read.

Cost

- App VM (instance i-07d38eb22d634abdb, 2 vCPU / 8 GB class) running 24/7: about $70 a month (docs estimate about $80 for 24/7 including disk and IP). It stays on through the VC review period; a daily uptime routine checks it.
- Elastic IP and the 30 GB disk bill even while stopped (about $6 a month together).
- Stop it with .\gearops-vm.ps1 stop after the review period (back up first).
- Paid demo runs (396 to 398) cost Anthropic API usage; the fine-tune demos run on CPU and cost only VM time.

8. Secrets and security

Where each secret lives and how it reaches the app

Name	Lives in	Reaches the app as	Used for
DB_PASSWORD	Secrets Manager gearops/vm	RIVET_DB_PASSWORD (orchestrator, spinner), MYSQL_ROOT_PASSWORD	Database login
JWT_SECRET	gearops/vm	RIVET_JWT_SECRET	Signing login tokens
MASTER_KEY	gearops/vm	RIVET_MASTER_KEY	Master encryption key
FF_KS_PASS	gearops/vm	FF_KS_PASS / FF_KEY_PASS	Opens the AES keystore that encrypts stored secrets
ADMIN_PASSWORD	gearops/vm	ADMIN_PASSWORD	First admin, only when there are no users
ENCRYPTION_KEY (optional)	gearops/vm	RIVET_ENCRYPTION_KEY	Legacy field encryption
GITHUB_CLIENT_SECRET	gearops/vm (being added)	RIVET_GITHUB_CLIENT_SECRET	GitHub sign-in handshake
GITHUB_APP_PRIVATE_KEY	gearops/vm (being added)	RIVET_GITHUB_APP_PRIVATE_KEY	Proves requests come from the mltorq-gearops app
ANTHROPIC_API_KEY (optional)	gearops/vm, and also a user secret in Settings > Secrets	RIVET_ANTHROPIC_API_KEY; ${secret:ANTHROPIC_API_KEY} in pipelines	LLM calls. The triage pipelines use the Settings secret.
REGISTRY_PASSWORD (optional)	gearops/vm	RIVET_REGISTRY_PASSWORD (spinner)	Private image registry
GEAROPS_DOMAIN, IMAGE_TAG, ADMIN_USERNAME, DOCKER_GID	SSM /gearops/config/	plain settings	Site name, image tag, etc.
User secrets, deploy target settings, encrypted task fields	MySQL, encrypted (ENCv1, AES-GCM)	decrypted by the spinner at run time, given to the job as SECRET_<field>	Customer keys, SSH keys, cloud credentials

The four-step flow (memorise this)

1. Secrets sit in AWS Secrets Manager (gearops/vm) and settings in SSM (/gearops/config/).
2. On deploy or boot, up.sh uses the VM's IAM role (no keys on disk) to read them and writes /run/gearops/gearops.env: in memory, mode 600, one KEY=value per line, never printed.
3. Docker Compose reads that file and passes values to containers as environment variables.
4. The orchestrator reads them through Spring placeholders, for example spring.datasource.password=${RIVET_DB_PASSWORD}. The app has no AWS code for secrets. A change in AWS takes effect only after up.sh runs again and the containers restart.

What is protected today

- Only ports 443 and 80 are open. No SSH port: shell access is through AWS SSM Session Manager.
- MySQL and Kafka are reachable only inside the Docker network.
- Stored secrets are encrypted with AES-GCM (ENCv1 tokens) using a keystore created on the VM; the browser never gets a secret back.
- Secret values are masked as **** in job logs (SecretMasker).
- Job containers: own network, AWS metadata endpoint blocked by an iptables rule (so a job cannot borrow the VM's AWS role), no Docker socket, no host folders, no app secrets, no-new-privileges, CPU/memory/process/time caps, removed after the run.
- The application properties files now read every secret from environment placeholders.

What is still weak (and the fix)

Weak spot	Why it matters	Fix
Old GitHub App private key body is still in git	orchestrator/src/main/resources/ application.properties (lines 92 to 117) and application-https.properties (24 lines) still contain the body of an RSA private key after the placeholder line, in the committed HEAD. It is likely the old Cicada-Integrator key.	Make sure that app and key are deleted/revoked on GitHub (deleting the app revokes its keys), remove the lines, and treat the history as leaked. Verify.
A GitHub personal access token in orchestrator/cadence.sql	A token inside git clone URLs in an old SQL dump (lines about 825 to 863).	Revoke it on GitHub; remove or scrub the file.
Old secrets in git history	Earlier versions of the properties files had literal passwords.	Rotate the Docker registry password and anything else that was ever committed.
Env file on the VM	All VM secrets sit in /run/gearops/gearops.env while it runs. Anyone with root on the VM can read it.	Later: fetch secrets at runtime from Secrets Manager, or use a secrets agent.
ECR login on the VM	docker login saves a registry token in the ubuntu user's Docker config on disk (it expires after 12 hours).	Use the ECR credential helper so nothing is stored.
Docker socket in the spinner	The spinner can start containers, which is root-equivalent on the VM. Fine for a single-owner demo box.	For shared or hosted use, run jobs on Kubernetes or ECS runners and remove the socket mount.
Kafka is plaintext	Only on the private Docker network.	TLS when components are split across machines.
Demo account password	The demo login is shared with reviewers.	Change it before sharing, and again after the review.

Say it: *Secrets never live in the code. On our VM they come from AWS Secrets Manager at boot through the machine's role. Customer secrets are encrypted in the database, only decrypted by the runner, handed to the job as environment variables and masked in logs. Jobs run in throwaway containers that cannot reach the cloud metadata service.*

9. GitHub integration end to end

A GitHub App is GitHub's way for a product to act on repos a customer chooses, with short-lived tokens instead of a personal password. GearOps now owns a new app.

Item	Value
App slug (public name)	mltorq-gearops
App ID	5555232
Client ID	Iv23liBmQqx4Yef0ibh9
Install link	https://github.com/apps/mltorq-gearops/installations/new
Setup URL to configure on GitHub	https://mltorq.com/api/github/app/setup
OAuth callback	https://mltorq.com/api/oauth/callback
Private key and client secret	In Secrets Manager gearops/vm only (never in git)

The flow

1. The customer clicks Install / Connect GitHub on the Repositories page. The UI sends the browser to the install link [GitHubConnect.jsx, RepoList.jsx].
2. On GitHub, the customer picks which repos MLTorq may see and confirms.
3. GitHub redirects to https://mltorq.com/api/github/app/setup?installation_id=...; Caddy strips /api and the orchestrator's GitHubAppSetupController handles /github/app/setup.
4. The orchestrator stores the installation_id against the user (userService.storeGitHubInstallation).
5. GitHubAppAuthService signs a JWT (a signed note saying "I am app 5555232") with the private key using RS256, valid for at most 10 minutes.
6. It sends that JWT to GitHub at /app/installations/{id}/access_tokens and gets back an installation token that lasts about an hour and only works on the chosen repos.
7. With that token it lists the repos (api.github.com/installation/repositories) and saves them, then redirects the browser to https://mltorq.com/repos?success=installed.
8. When a pipeline clones a private repo, the job service asks for a fresh installation token the same way, so no long-lived password is ever stored.

Current status and known gaps

- Code wiring for the new app is uncommitted on the laptop: 5 modified files (compose file, two UI files, two properties files) plus the new deploy/laptop/set-github-app.ps1. The private key and client secret still need to be loaded into AWS, then the orchestrator and UI deployed.
- The repo list is not paginated: only the first page from GitHub (30 by default) is saved.
- No webhook sync: if the customer adds or removes repos on GitHub later, MLTorq does not hear about it until setup runs again.
- Cloning a private repo on the Docker runner with the installation token is untested.
- Push / pull-request triggers are on the roadmap, not built.

10. Honest limitations and roadmap

Frame these as "MVP today, deliberate next steps". Every item is something you chose to defer, not something you missed.

Today	Next step
Single VM, one job at a time on the built-in runner.	Kubernetes and ECS runners already exist; scale out by adding runners.
Customer runners need network access from our spinner.	Runner agent inside the customer network that pulls work (roadmap).
Manual or API job starts.	Git push / PR and scheduled triggers, keeping secrets away from untrusted PRs.
Eval by exact labels and metrics.	Judge-model scoring.
Secrets in our own encrypted store.	Vault and AWS Secrets Manager backends.
Basic users and roles.	SSO, fine-grained RBAC, audit log, then SOC 2 (not started).
Container isolation.	Micro-VM runner isolation.
Cost from an editable price table.	Repo-aware cost prediction and real spot prices.
Azure and GCP deploy targets not all verified end to end.	Verify each with a real account.
Model serving needs Kubernetes.	Docker-runner serving, if customers ask.
Spinner unit test does not compile; images build with tests skipped.	Fix the test and run it in our own CI.
Fine-tune demo re-asks the training questions.	It proves the gate, not generalisation. Use a held-out set for real runs.

Business model (proposed, no prices set)

- Community: open-core, self-hosted engine, pipelines and runners.
- Enterprise: self-hosted licence with MLTorq features (fine-tune, routing, GPU cost), later SSO, RBAC and audit.
- Managed: GearOps runs the control plane, runners stay in the customer network (needs the runner agent).

11. Likely questions with short, honest answers

Q: Why not just use GitHub Actions or GitLab CI?

A: They run code builds well, but they have no model gates, no fine-tune with GPU cost and budget stop, and no "cheapest model that passes" step. You would glue five tools together. In MLTorq those are first-class tasks in the same run.

Q: Why not MLflow, Kubeflow or Weights & Biases?

A: They track experiments or run ML workflows, but they are not CI/CD: no security gates, no app deploys to 15+ targets. MLTorq covers both code and model in one pipeline.

Q: What exactly did you build yourself?

A: The whole RivetCI engine: the Spring Boot orchestrator and spinner, three runner kinds (Docker, Kubernetes, ECS), the task type system, secrets encryption and masking, 16 deploy types, four scanner gates, LoRA fine-tune, LLM routing and cost tracking, and the React UI.

Q: What is the moat?

A: The task type architecture. Every capability is a script plus a form, so new features inherit secrets, logs, gates, cost and runners for free, and customers can add their own. Plus it is independent and self-hosted, while point tools are being bought by larger platforms.

Q: Does customer data leave their account?

A: No, not when they run on their own runners. Jobs run on their Docker, Kubernetes or ECS. Today our control plane sends work to those runners; the runner agent will let them pull work from inside their network.

Q: How do you handle secrets?

A: Stored encrypted (AES-GCM), referenced by name, decrypted only by the runner, given to the job as environment variables and masked in logs. On our own VM, platform secrets come from AWS Secrets Manager at boot through the machine's role.

Q: How does the routing demo save money?

A: It tests each model on the same labelled tickets and picks the cheapest that clears the bar. In the demo Haiku tied Sonnet at 1.00, so a team would ship the much cheaper model. Be careful: we did not store a dollar figure for that run.

Q: Is the fine-tune real?

A: Yes, it is real LoRA training (transformers + peft), shown in CPU smoke mode with a 135M model so it is cheap and fast. GPU mode, S3 checkpoints, spot resume and a budget stop exist. The demo proves the gate, not model quality.

Q: Who are your customers? Any revenue?

A: Answer honestly with your real numbers. If none yet: "Pre-revenue; the MVP is live at mltorq.com and I am starting design-partner conversations with teams that fine-tune or route between models."

Q: Why you?

A: 20 years as a software engineer at principal and architect level in Java and cloud, hands-on with Jenkins, GitLab, CircleCI, MLflow and Kubeflow, and I built this engine solo. (Add your employers and one concrete accomplishment.)

Q: Why now?

A: Small and open models are cheaper, so teams switch models far more often, and each switch needs to be tested, gated and priced. Meanwhile the independent point tools (W&B, Langfuse, Helicone, promptfoo, Portkey) have been acquired by bigger platforms.

Q: How does it scale?

A: The control plane is small; jobs run on runners. Add Kubernetes or ECS runners for more parallel jobs. The single VM is our demo deployment, not the limit.

Q: What happens when a gate fails?

A: The script exits with an error, the task fails, and every later step, including deploy, is skipped. The job page shows exactly which gate blocked it and why.

Q: How do you deploy MLTorq itself?

A: Images are built and pushed to AWS ECR; on the VM a script pulls secrets from Secrets Manager, logs in to ECR and starts the stack with Docker Compose. Shell access is via SSM, with no SSH port.

Q: Biggest technical risk?

A: Running untrusted customer code. Today it is isolated in throwaway containers with no metadata access and resource caps; next is micro-VM isolation and the in-network runner agent.

Q: How does GitHub access work?

A: Through a GitHub App: the customer chooses repos, and we use one-hour tokens per installation instead of stored passwords.

Q: What would you do with the money?

A: Automate GPU provisioning and cost comparison, build the runner agent and Git triggers, and harden for enterprise (SSO, RBAC, audit). (Make the amount match the deck: the deck currently says >= $200k; your notes say $500k.)

Q: Is it open source?

A: Proposed open-core: the community engine is self-hosted; MLTorq ML features are the paid layer. A community export exists in the repo (README.community.md).

Q: What is RivetCI vs MLTorq vs GearOps?

A: GearOps is the company, MLTorq is the product, and RivetCI is the CI/CD engine inside it.

12. Glossary

Term	One-line meaning
CI/CD	Continuous integration and delivery: automatically build, test and deploy every change.
Pipeline / step / job	A chain of steps / one step in it / one run of it.
Task type	A recipe: script with placeholders plus form fields.
Gate	A step that stops the pipeline when a bar is not met.
Runner	The place job containers run: Docker, Kubernetes or ECS.
Container	A packaged, isolated process with its own files; thrown away after use.
Docker Compose	Starts several containers together from one YAML file.
Kubernetes (k8s)	A system that runs containers across many machines. EKS, AKS, GKE are the cloud versions.
AWS ECS / Fargate	AWS's own container service; Fargate runs containers without managing servers.
EC2 / VM	A rented virtual machine in AWS.
ECR	AWS's private store for container images.
Secrets Manager	AWS service that stores secrets encrypted.
SSM Parameter Store / Session Manager	AWS settings store / browser-or-CLI shell into a VM without SSH.
IAM role (instance role)	Permissions attached to the VM so it can call AWS without stored keys.
Caddy	Web server and HTTPS proxy with automatic certificates.
Kafka	A message queue between services.
Spring Boot	A Java framework for web services (orchestrator, spinner).
JWT	A signed token. Used for user logins and for proving the GitHub App's identity.
RS256	Signing with an RSA private key; anyone with the public key can check it.
AES-GCM / ENCv1	The encryption used for stored secrets / MLTorq's format for an encrypted value.
LoRA	A cheap way to fine-tune a model by training a small add-on instead of the whole model.
Eval set	Questions with known right answers, used to score a model.
Accuracy / macro F1	Share right / an average that is fairer when some labels are rare.
PSI (drift)	Population Stability Index: how much new data differs from training data. Over 0.2 = drift.
Spot capacity	Cheaper cloud machines that can be taken back at short notice.
SSE	Server-sent events: a live one-way feed from server to browser, used for logs.

13. Cheat sheet of commands (from the repo scripts)

Laptop (PowerShell), from C:\projects\gearops-deploy\deploy\laptop. All use AWS profile gearops.

```
.\gearops-vm.ps1 status        # VM state, IP, type
.\gearops-vm.ps1 start         # start the VM and wait
.\gearops-vm.ps1 stop          # stop the VM (run backup.sh on the VM first)
.\gearops-vm.ps1 ssm           # shell on the VM via SSM Session Manager
.\seed-demo-data.ps1           # run the free demo jobs as "demo" (asks password)
.\seed-demo-data.ps1 -IncludePaid   # also 396-398 (real Anthropic calls; type YES)
.\create-demo-user.ps1         # create the demo login
.\set-admin-password.ps1       # reset the admin password
.\set-github-app.ps1 -PemPath <path to .pem>   # load GitHub key + client secret into gearops/vm (new,
uncommitted)
.\apply-s3-logs.ps1            # S3 log bucket policy + metadata hop limit
```

Check secret key names without values (read-only):

```
aws secretsmanager describe-secret --secret-id gearops/vm --profile gearops --region us-east-1
```

On the VM (after .\gearops-vm.ps1 ssm), in the deploy folder ~/gearops/deploy:

```
./up.sh                       # fetch secrets + config, ECR login, pull, up -d
docker compose -f docker-compose.vm.yml --env-file /run/gearops/gearops.env ps
docker compose -f docker-compose.vm.yml --env-file /run/gearops/gearops.env logs --tail 100
orchestrator
./backup.sh                   # DB dump + keystore to /var/backups/gearops (and S3 if set)
sudo systemctl status gearops # the boot service that runs up.sh
```

Health check from anywhere:

```
https://mltorq.com/api/orchestrate/edition
```

Git (laptop): work on pipeline-run, then fast-forward develop; never touch main.

```
git checkout pipeline-run; git status; git log --oneline -5
git push origin pipeline-run; git push origin pipeline-run:develop   # fast-forward only
```

The last git line is the usual pattern, not a script in the repo.

14. Verify before the demo

1. Five merged pipelines: no merge found in the repo (the seed script still lists 391 to 401 separately). Check the live Pipelines page and pick which ids you show.
2. Deck slide 4 says Postgres; the stack runs MySQL 8. Fix the slide.
3. The ask: the deck says ">= $200k"; your notes say $500k. Make them match.
4. Old RSA private key text is still committed in orchestrator/src/main/resources/application.properties and application-https.properties. Confirm the old Cicada-Integrator app (and its keys) is deleted, then remove those lines.
5. Revoke the GitHub personal access token found in orchestrator/cadence.sql, and rotate the Docker registry password.
6. GitHub App: load the key and client secret, commit and deploy the 5 changed files, set the Setup URL on GitHub to https://mltorq.com/api/github/app/setup, then test one install end to end. Private-repo clone on the Docker runner is untested.
7. Orchestrator image is 271f40a, so the step-order fix (3ad8521) is not live in the API yet; the UI is.
8. Pipeline 399 deploy step: model serving needs a Kubernetes runner. Check what 399 shows on the Docker runner, and say "serving runs on Kubernetes" if asked.
9. Pipeline 401 needs the secret DEMO_TARGET_SSH_KEY under the demo user, and the demo target VM running; otherwise it fails at deploy.
10. Change the demo password before sharing it with reviewers.
11. No dollar cost is stored for the paid triage runs; do not quote savings in dollars unless you compute them.
12. No build-and-push script is committed; the build steps are in docs/deploy-ec2-plan.md. Image repo names there (gearops/<x>) differ from the compose file (gearops-<x>); the compose names are the real ones.
13. VM monthly cost: about $70 is from your notes; the docs estimate about $80 for 24/7 including disk and IP. Check the AWS bill if asked.
14. Deploy type count: the deck says 15 deploy targets; the repo has 16 deploy type scripts. Either is fine, just be consistent.